

PRODUCTION RAG ARCHITECTURE

AI Engineer Ready · Week 7 — Layered Design (Frontend → API → Services → RAG)

GOLDEN RULE `frontend/app.py` → `api/routes.py` → `services/*_service.py` → `rag/` building blocks — each layer only calls the one below it.

1. Why layers at all

A notebook RAG script mixes UI, orchestration, and retrieval logic in one cell. A production system splits those concerns into layers so a bug is **findable**: is the building block wrong (e.g. `mmr.py`'s λ value), or is the orchestration wrong (e.g. steps called in the wrong order)? Each layer only ever calls the layer directly below it — never skips one, never reaches sideways.

Layer	Folder	Responsibility	Must NOT contain
Frontend	<code>frontend/app.py</code>	Streamlit UI: upload widget, question box, render answer + metrics	RAG logic, direct FAISS/LLM calls
API	<code>api/routes.py</code> , <code>health.py</code>	HTTP contract: POST <code>/upload</code> , POST <code>/retrieve</code> , GET <code>/health</code>	Business logic — just calls a service
Services	<code>services/*_service.py</code>	Orchestrators: <code>ingestion_service</code> , <code>retrieval_service</code> — the step order	Low-level RAG mechanics
RAG building blocks	<code>rag/*.py</code>	loader, splitter, embeddings, <code>faiss_store</code> , <code>mmr</code> , compression, <code>multi_query_retriever</code> , <code>deduplicator</code> , <code>chain</code> , <code>llm</code>	Knowledge of HTTP, Streamlit, or request/response shape

2. The request flow, end to end

- User clicks **Get Answer** in Streamlit → `frontend/app.py`
- Frontend makes a real **HTTP call** to POST `/retrieve` — not a Python import
- `api/routes.py` calls `services/retrieval_service.retrieve_answer(question, model, retriever_type)`
- The service picks a retrieval strategy, calls `rag/` building blocks to fetch chunks, dedupes, then generates
- The service also calls `evaluation/ragas_eval.py` to score the answer (7 RAGAS / LLM-judge metrics)
- Service returns (`answer`, `answer_texts`, `evaluation_metric`) up through the API to the frontend

✓ Why HTTP, not an import

The frontend calling the backend over HTTP — even when both run in the same container — is deliberate. It mirrors how a real external client would talk to your API, and it's what makes the backend independently testable, scalable, and swappable for a different frontend later.

3. Diagnosing a bug by layer

Symptom	Layer to check first
Streamlit shows a raw traceback or crashes	Frontend — likely a <code>None</code> from a backend call not being handled
404 or 422 on a request that should work	API — route path, method, or request schema mismatch
Right answer sometimes, wrong step order other times (e.g. re-ingest doubles chunk count)	Services — orchestration bug

Symptom	Layer to check first
Answer is wrong even though the pipeline ran without errors	RAG building blocks — retrieval strategy or prompt/llm.py

■ **Pitfall — Layer-skipping**

The moment routes.py starts importing from rag/ directly, or app.py starts calling a service function in-process, the whole point of the layering is gone — you can no longer tell which layer broke just from the symptom. Keep every call one layer down, no exceptions.

4. Interview Prep

Q: Why split retrieval logic into services/ vs. rag/ instead of one big file?

A: Services encode the *order* of steps for a use case (ingest, retrieve); rag/ holds reusable, independently testable building blocks. Mixing them means every new use case reimplements low-level logic, and every low-level bugfix risks breaking orchestration.

Q: Why does the frontend call the backend over HTTP instead of importing it directly?

A: It enforces the same boundary a real external client would face, keeps the backend independently deployable and testable, and prevents the UI layer from silently depending on RAG internals.

Q: How would you test this architecture layer by layer?

A: Unit-test rag/ building blocks in isolation (e.g. mmr.py with a fixed embedding set), test services/ with the rag/ layer mocked to check step order, and test api/routes.py with the service mocked to check the HTTP contract (status codes, schema).